

1.INTRODUCTION

In the digital era, the hiring landscape has undergone a significant transformation. With the widespread use of Applicant Tracking Systems (ATS) by employers to streamline the recruitment process, the traditional method of manually reviewing resumes has become largely obsolete. ATS are designed to filter out unsuitable applications before they even reach a human recruiter, often basing decisions on keyword matching, formatting, and relevance to job descriptions. While efficient from an employer's standpoint, this system presents a substantial barrier for job seekers who are unfamiliar with how these technologies operate.

Most candidates struggle to craft resumes that meet ATS criteria, often due to poor keyword usage, inconsistent formatting, and lack of personalization for different job roles. Additionally, common resume builders focus more on design than content optimization, failing to provide intelligent assistance or feedback mechanisms that can help candidates improve their resumes. This results in the rejection of potentially qualified candidates who may have the right skills but fail to present them effectively in their resumes.

To address this gap, the integration of Artificial Intelligence (AI), specifically Natural Language Processing (NLP) and Machine Learning (ML), into the resume-building process offers a revolutionary solution. By using AI to analyze job descriptions, identify key terms, and provide content suggestions, candidates can craft more targeted and impactful resumes. These technologies can also evaluate grammar, structure, and keyword density, offering real-time feedback and recommendations.

The system proposed in this study aims to simplify and enhance the resume creation process for job seekers by developing an intelligent, ATS-aware platform. It not only generates resumes based on user inputs and job-related keywords but also adapts to industry-specific requirements. Additionally, it recommends suitable job roles based on the user's skills and experience, increasing their chances of getting noticed by recruiters.

Another key feature is the ability to provide detailed insights into how well a resume aligns with a particular job post. This allows users to iteratively improve their applications before submission. Unlike traditional resume tools, this system leverages AI to enhance both content quality and presentation, improving the chances of success in automated screenings.

In summary, the proposed AI-powered resume system addresses major shortcomings in existing tools by automating keyword optimization, template selection, and content evaluation. It empowers job seekers to present themselves more effectively and helps employers identify high-quality candidates more efficiently. This research aims to set the foundation for future developments in AI-based career services and intelligent employment platforms.

Objectives, Scope, and Goals (Point-wise):

Objectives:

- To develop an AI-based system that auto-generates ATS-compliant resumes.
- To provide real-time feedback on resume grammar, structure, and keyword usage.
- To match resume content with industry-specific job requirements using NLP.
- To recommend job roles based on user-provided skills and experience.

Scope:

- Focuses on improving resume creation for job seekers across various domains.
- Utilizes AI techniques such as NLP and ML for automation and intelligence.
- Supports both technical and non-technical users with minimal manual input.
- Designed to work across various industries and job platforms.

Goals:

- Enhance candidate visibility in ATS-based screenings.
- Automate content and template customization for job applications.
- Improve the hiring experience for recruiters and job seekers.
- Lay the groundwork for future AI-driven career advisory systems.

2.LITERATURE SURVEY

Recent years have witnessed the rapid integration of Artificial Intelligence in Human Resource Management, particularly in the recruitment and selection process. One of the most critical advancements has been the adoption of ATS systems, which automatically filter resumes based on predefined criteria. Studies such as those by Cappelli (2019) highlight that over 75% of resumes submitted online are rejected by ATS before reaching human recruiters. This has shifted the focus of job seekers toward optimizing resumes to meet algorithmic standards.

Despite this trend, traditional resume-building tools fall short in providing intelligent support for candidates. Research by Nikolaou (2020) found that while online resume templates are widely used, most of them are not designed to be ATS-compatible. Furthermore, these tools often emphasize design aesthetics over content relevance, leading to mismatches between resume data and job role requirements.

NLP and ML technologies have shown significant promise in addressing this issue. According to a 2022 report by the Society for Human Resource Management (SHRM), AI-based resume parsers can analyze textual content and recommend improvements in real-time. Such systems utilize techniques like named entity recognition, part-of-speech tagging, and sentiment analysis to better understand resume context. This allows for the automatic identification of skills, experiences, and roles that match specific job postings.

Several AI-driven platforms like Rezi, Zety, and JobScan already incorporate partial keyword analysis and feedback systems. However, these are often subscription-based and lack transparency in their scoring mechanisms. A comparative study by Bhandari et al. (2021) reveals that most users are unaware of how ATS evaluates their resumes, leading to confusion and low confidence in the submission process.

Additionally, AI in recruitment isn't limited to resume building. Machine learning algorithms are being used for job-role recommendations based on skill clusters and experience histories. Research by Dinevski and Kovačič (2020) introduced a smart employment advisory system that matched job seekers with vacancies using collaborative filtering and clustering algorithms. Their results indicated a 30% increase in successful placements when job suggestions were aligned with users' skill profiles.

Ethical concerns, however, have also emerged with the use of AI in hiring. Some researchers argue that AI-based resume creation could potentially fabricate exaggerated profiles, misleading recruiters. Binns et al. (2018) discuss the need for transparency, fairness, and accountability in AI models used in recruitment. Therefore, a balance between automated enhancement and authentic representation must be maintained.

3.SYSTEM ANALYSIS

Existing System, Disadvantages, Proposed System, Advantages:

Existing System:

- Manual or semi-automated resume builders.
- Template-based editors with limited intelligence.
- No real-time content feedback or ATS optimization.

Disadvantages:

- Candidates unknowingly submit non-ATS-compliant resumes.
- Higher rejection rates due to poor keyword alignment.
- No recommendations for improving resume quality.

Proposed System:

- AI and NLP-based automated resume builder.
- Intelligent keyword extraction from job descriptions.
- Real-time feedback on grammar, content, and structure.

Advantages:

- Provides personalized suggestions to improve content.
- Reduces manual effort and improves efficiency.
- Empowers job seekers with intelligent guidance and role targeting.

4.SYSTEM REQUIREMENTS

The project involved analyzing the design of few applications so as to make the application more users friendly. To do so, it was really important to keep the navigations from one screen to the other well ordered and at the same time reducing the amount of typing the user needs to do. Inorder to make the application more accessible, the browser version had to be chosen so that it is compatible with most of the Browsers.

REQUIREMENT SPECIFICATION:

Functional Requirements:

- Graphical User interface with the User.

Software Requirements:

For developing the application the following are the Software Requirements:

1. Python3.8.10

Operating Systems supported:

1. Windows family
2. Windows XP
3. Windows 8 or 10 or 11
4. TXT File or CSV

Technologies and Languages used to Develop:

1. Python

Debugger and Emulator:

- Any Browser (Particularly Chrome)

Hardware Requirements:

For developing the application, the following are the Hardware Requirements:

- Processor: Pentium IV or higher
- RAM: 256 MB

5.FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is estial.

Three key considerations involved in the feasibility analysis are,

- ◆ ECONOMICAL FEASIBILITY
- ◆ TECHNICAL FEASIBILITY
- ◆ SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system.

6.SYSTEM STUDY

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format

- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

7.SYSTEM DESIGN

INPUT AND OUTPUT DESIGN:

INPUT DESIGN:

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

OBJECTIVES

1.Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3.When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow

OUTPUT DESIGN:

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system

through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

8.SOFTWARE ENVIRONMENT

1.1 PYTHON

Python is a **high-level, interpreted, interactive and object-oriented scripting language**. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

1.2 History of Python:

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

1.3 Python Features:

Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.

- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Python has a big list of good features:

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

Operators in Python



2.1 ARITHMETIC OPERATORS

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 30$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -10$
* Multiplication	Multiplies values on either side of the operator	$a * b = 200$
/ Division	Divides left hand operand by right hand operand	$b / a = 2$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 0$

** Exponent	Performs exponential (power) calculation on operators	$a^{**}b = 10$ to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	$9//2 = 4$ and $9.0//2.0 = 4.0$, $-11//3 = -4$, $-11.0//3 = -4.0$

2.2ASSIGNMENT OPERATOR

Operator	Description	Example
=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of $a + b$ into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	$c -= a$ is equivalent to $c = c - a$

*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	$c *= a$ is equivalent to $c = c * a$
/= Divide AND	It divides left operand with the right operand and assign the result to left operand	$c /= a$ is equivalent to $c = c / a$ $c /= a$ is equivalent to $c = c / a$

%= Modulus AND	It takes modulus using two operands and assign the result to left operand	$c \% a$ is equivalent to $c = c \% a$
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	$c ** a$ is equivalent to $c = c ** a$
//= Floor Division	It performs floor division on operators and assign value to the left operand	$c // a$ is equivalent to $c = c // a$

2.3 IDENTITY OPERATOR

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y , here is results in 1 if $id(x)$ equals $id(y)$.

is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y)
--------	---	--

2.4 COMPARISON OPERATOR

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) (means 0000 1100)
Binary OR	It copies a bit if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.

<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)

2.5 LOGICAL OPERATOR

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

Python Operators Precedence:

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction

>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= < > >=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

3.1 LIST

The list is a most versatile data type available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets.

Basic List Operations:

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

Python Expression	Results	Description
-------------------	---------	-------------

<code>len([1, 2, 3])</code>	3	Length
<code>[1, 2, 3] + [4, 5, 6]</code>	<code>[1, 2, 3, 4, 5, 6]</code>	Concatenation
<code>['Hi!'] * 4</code>	<code>['Hi!', 'Hi!', 'Hi!', 'Hi!']</code>	Repetition
<code>3 in [1, 2, 3]</code>	True	Membership
<code>for x in [1, 2, 3]: print x,</code>	1 2 3	Iteration

Built-in List Functions & Methods:

Python includes the following list functions –

SN	Function with Description
1	<u><code>cmp(list1, list2)</code></u> Compares elements of both lists.
2	<u><code>len(list)</code></u> Gives the total length of the list.
3	<u><code>max(list)</code></u> Returns item from the list with max value.
4	<u><code>min(list)</code></u> Returns item from the list with min value.
5	<u><code>list(seq)</code></u> Converts a tuple into list.

Python includes following list methods

SN	Methods with Description
1	<u>list.append(obj)</u> Appends object obj to list
2	<u>list.count(obj)</u> Returns count of how many times obj occurs in list
3	<u>list.extend(seq)</u> Appends the contents of seq to list
4	<u>list.index(obj)</u> Returns the lowest index in list that obj appears
5	<u>list.insert(index, obj)</u> Inserts object obj into list at offset index
6	<u>list.pop(obj=list[-1])</u> Removes and returns last object or obj from list
7	<u>list.remove(obj)</u> Removes object obj from list
8	<u>list.reverse()</u> Reverses objects of list in place
9	<u>list.sort([func])</u> Sorts objects of list, use compare function if given

3.2 TUPLES

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally we can put these comma-separated values between parentheses also. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
tup2 = (1, 2, 3, 4, 5 );  
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing –

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value-

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

Accessing Values in Tuples:

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
tup2 = (1, 2, 3, 4, 5, 6, 7 );  
print "tup1[0]: ", tup1[0]  
print "tup2[1:5]: ", tup2[1:5]
```

When the code is executed, it produces the following result –

```
tup1[0]: physics  
tup2[1:5]: [2, 3, 4, 5]
```

Updating Tuples:

Tuples are immutable which means you cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples as the following example demonstrates –

```
tup1 = (12, 34.56);
```

```
tup2 = ('abc', 'xyz');  
  
tup3 = tup1 + tup2;  
  
print tup3
```

When the above code is executed, it produces the following result –

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements:

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the **del** statement. For example:

```
tup = ('physics', 'chemistry', 1997, 2000);  
  
print tup  
  
del tup;  
  
print "After deleting tup : "  
  
print tup
```

Basic Tuples Operations:

Python Expression	Results	Description
len((1, 2, 3))	3	Length
(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)	Concatenation
('Hi!') * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	Repetition
3 in (1, 2, 3)	True	Membership

for x in (1, 2, 3): print x,	1 2 3	Iteration
------------------------------	-------	-----------

Built-in Tuple Functions:

SN	Function with Description
1	cmp(tuple1, tuple2) :Compares elements of both tuples.
2	len(tuple) :Gives the total length of the tuple.
3	max(tuple) :Returns item from the tuple with max value.
4	min(tuple) :Returns item from the tuple with min value.
5	tuple(seq) :Converts a list into tuple.

3.2 DICTIONARY

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Accessing Values in Dictionary:

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value.
Result –

```
dict['Name']: Zara
dict['Age']: 7
```

Updating Dictionary:

We can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example –


```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

dict['Age'] = 8; # update existing entry

dict['School'] = "DPS School"; # Add new entry

print "dict['Age']: ", dict['Age']

print "dict['School']: ", dict['School']
```

Result –

```
dict['Age']: 8
dict['School']: DPS School
```

Delete Dictionary Elements:

We can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the **del** statement. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

del dict['Name']; # remove entry with key 'Name'

dict.clear();    # remove all entries in dict

del dict ;      # delete entire dictionary

print "dict['Age']: ", dict['Age']

print "dict['School']: ", dict['School']
```

Built-in Dictionary Functions & Methods –

Python includes the following dictionary functions –

SN	Function with Description
----	---------------------------

1	<u>cmp(dict1, dict2)</u> Compares elements of both dict.
2	<u>len(dict)</u> Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
3	<u>str(dict)</u> Produces a printable string representation of a dictionary
4	<u>type(variable)</u> Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Python includes following dictionary methods –

SN	Methods with Description
1	dict.clear(): Removes all elements of dictionary <i>dict</i>
2	dict. Copy(): Returns a shallow copy of dictionary <i>dict</i>
3	dict.fromkeys(): Create a new dictionary with keys from seq and values <i>set</i> to <i>value</i> .
4	dict.get(key, default=None): For <i>key</i> key, returns value or default if key not in dictionary
5	dict.has_key(key): Returns <i>true</i> if key in dictionary <i>dict</i> , <i>false</i> otherwise
6	dict.items(): Returns a list of <i>dict</i> 's (key, value) tuple pairs

7	dict.keys(): Returns list of dictionary dict's keys
8	dict.setdefault(key, default=None): Similar to get(), but will set dict[key]=default if <i>key</i> is not already in dict
9	dict.update(dict2): Adds dictionary <i>dict2</i> 's key-values pairs to <i>dict</i>
10	dict.values(): Returns list of dictionary <i>dict</i> 's values

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing. Python gives you many built-in functions like print(), etc. but you can also create your own functions. These functions are called *user-defined functions*.

Defining a Function:

Simple rules to define a function in Python.

- Function blocks begin with the keyword def followed by the function name and parentheses (()).
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon (:) and is indented.
- The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

```
def functionname( parameters ):
    "function_docstring"
    function_suite
    return [expression]
```

Calling a Function:

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call printme() function –

```
# Function definition is here

def printme( str ):

    "This prints a passed string into this function"

    print str

    return;

# Now you can call printme function

printme("I'm first call to user defined function!")
```

When the above code is executed, it produces the following result –

```
I'm first call to user defined function!

Again second call to the same function
```

Function Arguments:

You can call a function by using the following types of formal arguments:

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Scope of Variables:

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python –

Global variables

Local variables

Global vs. Local variables:

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example –

```
total = 0; # This is global variable.

# Function definition is here

def sum( arg1, arg2 ):

    # Add both the parameters and return them."

    total = arg1 + arg2; # Here total is local variable.

    print "Inside the function local total : ", total

    return total;

sum( 10, 20 );

print "Outside the function global total : ", total
```

Result –

```
Inside the function local total : 30

Outside the function global total : 0
```

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference. Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

The import Statement

The *import* has the following syntax:

```
import module1[, module2[,... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module *support.py*, you need to put the following command at the top of the script. A module is loaded only once, regardless of the number of times it is imported. This prevents the module execution from happening over and over again if multiple imports occur.

Packages in Python:

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and sub packages and sub-sub packages.

Consider a file *Pots.py* available in *Phone* directory. This file has following line of source code –

```
def Pots():  
    print "I'm Pots Phone"
```

Similar way, we have another two files having different functions with the same name as above –

- *Phone/Isdn.py* file having function *Isdn()*
- *Phone/G3.py* file having function *G3()*

Now, create one more file *__init__.py* in *Phone* directory –

- *Phone/__init__.py*

To make all of your functions available when you've imported *Phone*, to put explicit import statements in *__init__.py* as follows –

```
from Pots import Pots  
  
from Isdn import Isdn  
  
from G3 import G3
```

After you add these lines to `__init__.py`, you have all of these classes available when you import the Phone package.

```
# Now import your Phone Package.
```

```
import Phone
```

```
Phone.Pots()
```

```
Phone.Isdn()
```

```
Phone.G3()
```

RESULT:

```
I'm Pots Phone
```

```
I'm 3G Phone
```

```
I'm ISDN Phone
```

In the above example, we have taken example of a single functions in each file, but you can keep multiple functions in your files. You can also define different Python classes in those files and then you can create your packages out of those classes.

This chapter covers all the basic I/O functions available in Python.

Printing to the Screen:

The simplest way to produce output is using the *print* statement where you can pass zero or more expressions separated by commas. This function converts the expressions you pass into a string and writes the result to standard output as follows –

```
print "Python is really a great language,", "isn't it?"
```

Result:

```
Python is really a great language, isn't it?
```

Reading Keyboard Input:

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard. These functions are –

- `raw_input`
- `input`

The `raw_input` Function:

The `raw_input([prompt])` function reads one line from standard input and returns it as a string (removing the trailing newline).

```
str = raw_input("Enter your input: ");
print "Received input is : ", str
```

This prompts you to enter any string and it would display same string on the screen. When I typed "Hello Python!", its output is like this –

```
Enter your input: Hello Python
Received input is : Hello Python
```

The `input` Function:

The `input([prompt])` function is equivalent to `raw_input`, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

```
str = input("Enter your input: ");
print "Received input is : ", str
```

This would produce the following result against the entered input –

```
Enter your input: [x*5 for x in range(2,10,2)]
Received input is : [10, 20, 30, 40]
```

Opening and Closing Files:

Until now, you have been reading and writing to the standard input and output. Now, we will see how to use actual data files.

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a **file** object.

The open Function:

Before you can read or write a file, you have to open it using Python's built-in *open()* function. This function creates a **file** object, which would be utilized to call other support methods associated with it.

Syntax

```
file object = open(file_name [, access_mode][, buffering])
```

Here are parameter details:

- **file_name:** The file_name argument is a string value that contains the name of the file that you want to access.
- **access_mode:** The access_mode determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

Here is a list of the different modes of opening a file –

Modes	Description
R	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
Rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.

rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
W	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
Wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
A	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
Ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

The file Object Attributes:

Once a file is opened and you have one *file* object, you can get various information related to that file.

Here is a list of all attributes related to file object:

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.

This produces the following result –

```
Name of the file: foo.txt
```

```
Closed or not : False
```

```
Opening mode : wb
```

```
Softspace flag : 0
```

The close() Method

The close() method of a *file* object flushes any unwritten information and closes the file object, after which no more writing can be done. Python automatically closes a file when the reference object of a file is reassigned to another file. It is a good practice to use the close() method to close a file.

Syntax:

```
fileObject.close();
```

Example

```
# Open a file
```

```
fo = open("foo.txt", "wb")
```

```
print "Name of the file: ", fo.name
```

```
# Close opened file
```

```
fo.close()
```

Result –

```
Name of the file: foo.txt
```

Reading and Writing Files

The *file* object provides a set of access methods to make our lives easier. We would see how to use *read()* and *write()* methods to read and write files.

The write() Method

The *write()* method writes any string to an open file. It is important to note that Python strings can have binary data and not just text. The *write()* method does not add a newline character ('\n') to the end of the string **Syntax**

```
fileObject.write(string);
```

Here, passed parameter is the content to be written into the opened file.

The above method would create *foo.txt* file and would write given content in that file and finally it would close that file. If you would open this file, it would have following content.

```
Python is a great language.
```

```
Yeah its great!!
```

The read() Method

The *read()* method reads a string from an open file. It is important to note that Python strings can have binary data. apart from text data.

Syntax:

```
fileObject.read([count]);
```

Here, passed parameter is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if *count* is missing, then it tries to read as much as possible, maybe until the end of file.

Example:

Let's take a file *foo.txt*, which we created above.

```
# Open a file
```

```
fo = open("foo.txt", "r+")

str = fo.read(10);

print "Read String is : ", str

# Close opened file

fo.close()
```

This produces the following result –

```
Read String is : Python is
```

File Positions:

The *tell()* method tells you the current position within the file; in other words, the next read or write will occur at that many bytes from the beginning of the file.

32

The *seek(offset[, from])* method changes the current file position. The *offset* argument indicates the number of bytes to be moved. The *from* argument specifies the reference position from where the bytes are to be moved.

If *from* is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position.

Example:

Let us take a file *foo.txt*, which we created above.

```
# Open a file

fo = open("foo.txt", "r+")

str = fo.read(10);

print "Read String is : ", str


# Check current position

position = fo.tell();
```

37

```
print "Current file position : ", position

# Reposition pointer at the beginning once again

position = fo.seek(0, 0);

str = fo.read(10);

print "Again read String is : ", str

# Close opened file

fo.close()
```

This produces the following result –

```
Read String is : Python is

Current file position : 10

Again read String is : Python is
```

Renaming and Deleting Files:

Python **os** module provides methods that help you perform file-processing operations, such as renaming and deleting files.

To use this module you need to import it first and then you can call any related functions.

The `rename()` Method

The `rename()` method takes two arguments, the current filename and the new filename.

Syntax

```
os.rename(current_file_name, new_file_name)
```

Example:

Following is the example to rename an existing file *test1.txt*:

```
import os
```

```
# Rename a file from test1.txt to test2.txt
```

```
os.rename( "test1.txt", "test2.txt" )
```

The `remove()` Method

You can use the `remove()` method to delete files by supplying the name of the file to be deleted as the argument.

Syntax:

```
os.remove(file_name)
```

Example:

Following is the example to delete an existing file `test2.txt` –

```
#!/usr/bin/python
```

```
import os
```

```
# Delete file test2.txt
```

```
os.remove("text2.txt")
```

Directories in Python:

All files are contained within various directories, and Python has no problem handling these too. The `os` module has several methods that help you create, remove, and change directories.

The `mkdir()` Method

You can use the `mkdir()` method of the `os` module to create directories in the current directory. You need to supply an argument to this method which contains the name of the directory to be created.

Syntax:

```
os.mkdir("newdir")
```

Example:

Following is the example to create a directory `test` in the current directory –

```
#!/usr/bin/python
```

```
import os
```

```
# Create a directory "test"
```

```
os.mkdir("test")
```

The chdir() Method

You can use the *chdir()* method to change the current directory. The *chdir()* method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax:

```
os.chdir("newdir")
```

Example:

Following is the example to go into "/home/newdir" directory –

```
#!/usr/bin/python
```

```
import os
```

```
# Changing a directory to "/home/newdir"
```

```
os.chdir("/home/newdir")
```

The getcwd() Method

The *getcwd()* method displays the current working directory.

Syntax:

```
os.getcwd()
```

Example:

Following is the example to give current directory –

```
import os
```

```
# This would give location of the current directory
```



```
os.getcwd()
```

The `rmdir()` Method

The `rmdir()` method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax:

```
os.rmdir('dirname')
```

Example:

Following is the example to remove `"/tmp/test"` directory. It is required to give fully qualified name of the directory, otherwise it would search for that directory in the current directory.

```
import os

# This would remove "/tmp/test" directory.

os.rmdir( "/tmp/test" )
```

File & Directory Related Methods:

There are three important sources, which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows –

- File Object Methods: The *file* object provides functions to manipulate files.
- OS Object Methods: This provides methods to process files as well as directories.

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them –

- **Exception Handling**: This would be covered in this tutorial. Here is a list standard Exceptions available in Python: Standard Exceptions.
- **Assertions**: This would be covered in Assertions in Python

List of Standard Exceptions –

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.
AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
ImportError	Raised when an import statement fails.

KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.
IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
IOError	Raised for operating system-related errors.
SyntaxError	Raised when there is an error in Python syntax.
IndentationError	Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.

RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

What is Exception?

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

Handling an exception

If you have some *suspicious* code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block. After the try: block, include an **except:** statement, followed by a block of code which handles the problem as elegantly as possible.

The Python standard for database interfaces is the Python DB-API. Most Python database interfaces adhere to this standard.

You can choose the right database for your application. Python Database API supports a wide range of database servers such as –

- GadFly
- mSQL
- MySQL
- PostgreSQL
- Microsoft SQL Server 2000
- Informix

Virtual Environments and Packages

Introduction

Python applications will often use packages and modules that don't come as part of the standard library. Applications will sometimes need a specific version of a library, because the application may require that a particular bug has been fixed or the application may be written using an obsolete version of the library's interface.

This means it may not be possible for one Python installation to meet the requirements of every application. If application A needs version 1.0 of a particular module but application B needs version 2.0, then the requirements are in conflict and installing either version 1.0 or 2.0 will leave one application unable to run.

The solution for this problem is to create a virtual environment, a self-contained directory tree that contains a Python installation for a particular version of Python, plus a number of additional packages.

Different applications can then use different virtual environments. To resolve the earlier example of conflicting requirements, application A can have its own virtual environment with version 1.0 installed while application B has another virtual environment with version 2.0. If application B requires a library be upgraded to version 3.0, this will not affect application A's environment.

Creating Virtual Environments

The module used to create and manage virtual environments is called `venv`. `venv` will usually install the most recent version of Python that you have available. If you have multiple versions of Python on your system, you can select a specific Python version by running `python3` or whichever version you want.

To create a virtual environment, decide upon a directory where you want to place it, and run the `venv` module as a script with the directory path:

```
python3 -m venv tutorial-env
```

This will create the `tutorial-env` directory if it doesn't exist, and also create directories inside it containing a copy of the Python interpreter, the standard library, and various supporting files.

A common directory location for a virtual environment is `.venv`. This name keeps the directory typically hidden in your shell and thus out of the way while giving it a name that explains why the directory exists. It also prevents clashing with `.env` environment variable definition files that some tooling supports.

Once you've created a virtual environment, you may activate it.

On Windows, run:

```
tutorial-env\Scripts\activate.bat
```

On Unix or MacOS, run:

```
source tutorial-env/bin/activate
```

(This script is written for the bash shell. If you use the csh or fish shells, there are alternate activate.csh and activate.fish scripts you should use instead.)

Activating the virtual environment will change your shell's prompt to show what virtual environment you're using, and modify the environment so that running python will get you that particular version and installation of Python. For example:

```
$ source ~/envs/tutorial-env/bin/activate
```

```
(tutorial-env) $ python
```

```
Python 3.5.1 (default, May 6 2016, 10:59:36)
```

```
>>> import sys
```

```
>>> sys.path
```

```
['', '/usr/local/lib/python35.zip', ...,
```

```
'~/envs/tutorial-env/lib/python3.5/site-packages']
```

```
>>>
```

12.3. Managing Packages with pip

You can install, upgrade, and remove packages using a program called pip. By default pip will install packages from the Python Package Index, <<https://pypi.org>>. You can browse the Python Package Index by going to it in your web browser, or you can use pip's limited search feature:

```
(tutorial-env) $ pip search astronomy
```

```
skyfield          - Elegant astronomy for Python
```

```
gary              - Galactic astronomy and gravitational dynamics.
```

```
novas             - The United States Naval Observatory NOVAS astronomy library
```

astroobs - Provides astronomy ephemeris to plan telescope observations

PyAstronomy - A collection of astronomy related tools for Python.

pip has a number of subcommands: “search”, “install”, “uninstall”, “freeze”, etc. (Consult the Installing Python Modules guide for complete documentation for pip.)

You can install the latest version of a package by specifying a package’s name:

```
(tutorial-env) $ pip install novas
```

Collecting novas

Downloading novas-3.1.1.3.tar.gz (136kB)

Installing collected packages: novas

Running setup.py install for novas

Successfully installed novas-3.1.1.3

You can also install a specific version of a package by giving the package name followed by == and the version number:

```
(tutorial-env) $ pip install requests==2.6.0
```

Collecting requests==2.6.0

Using cached requests-2.6.0-py2.py3-none-any.whl

Installing collected packages: requests

Successfully installed requests-2.6.0

If you re-run this command, pip will notice that the requested version is already installed and do nothing. You can supply a different version number to get that version, or you can run `pip install --upgrade` to upgrade the package to the latest version:

```
(tutorial-env) $ pip install --upgrade requests
```

Collecting requests

Installing collected packages: requests

Found existing installation: requests 2.6.0

Uninstalling requests-2.6.0:

Successfully uninstalled requests-2.6.0

Successfully installed requests-2.7.0

pip uninstall followed by one or more package names will remove the packages from the virtual environment.

pip show will display information about a particular package:

```
(tutorial-env) $ pip show requests
```

Metadata-Version: 2.0

Name: requests

Version: 2.7.0

Summary: Python HTTP for Humans.

Home-page: <http://python-requests.org>

Author: Kenneth Reitz

Author-email: me@kennethreitz.com

License: Apache 2.0

Location: /Users/akuchling/envs/tutorial-env/lib/python3.4/site-packages

Requires:

pip list will display all of the packages installed in the virtual environment:

```
(tutorial-env) $ pip list
```

novas (3.1.1.3)

numpy (1.9.2)

pip (7.0.3)

requests (2.7.0)

setuptools (16.0)

pip freeze will produce a similar list of the installed packages, but the output uses the format that pip install expects. A common convention is to put this list in a requirements.txt file:

```
(tutorial-env) $ pip freeze > requirements.txt
```

```
(tutorial-env) $ cat requirements.txt
```

```
novas==3.1.1.3
```

```
numpy==1.9.2
```

```
requests==2.7.0
```

The requirements.txt can then be committed to version control and shipped as part of an application. Users can then install all the necessary packages with install -r:

```
(tutorial-env) $ pip install -r requirements.txt
```

```
Collecting novas==3.1.1.3 (from -r requirements.txt (line 1))
```

```
Collecting numpy==1.9.2 (from -r requirements.txt (line 2))
```

```
Collecting requests==2.7.0 (from -r requirements.txt (line 3))
```

```
Installing collected packages: novas, numpy, requests
```

```
Running setup.py install for novas
```

```
Successfully installed novas-3.1.1.3 numpy-1.9.2 requests-2.7.0
```

pip has many more options. Consult the Installing Python Modules guide for complete documentation for pip. When you've written a package and want to make it available on the Python Package Index, consult the Distributing Python Modules guide.

Cross Platform

Platform. Architecture (executable=sys.executable, bits="", linkage=")

Queries the given executable (defaults to the Python interpreter binary) for various architecture information.

Returns a tuple (bits, linkage) which contain information about the bit architecture and the linkage format used for the executable. Both values are returned as strings.

Values that cannot be determined are returned as given by the parameter presets. If bits is given as "", the sizeof(pointer) (or sizeof(long) on Python version < 1.5.2) is used as indicator for the supported pointer size.

The function relies on the system's file command to do the actual work. This is available on most if not all Unix platforms and some non-Unix platforms and then only if the executable points to the Python interpreter. Reasonable defaults are used when the above needs are not met.

Note On Mac OS X (and perhaps other platforms), executable files may be universal files containing multiple architectures.

To get at the “64-bitness” of the current interpreter, it is more reliable to query the sys.maxsize attribute:

```
is_64bits = sys.maxsize > 2**32
```

platform.machine ()

Returns the machine type, e.g. 'i386'. An empty string is returned if the value cannot be determined.

platform.node ()

Returns the computer's network name (may not be fully qualified!). An empty string is returned if the value cannot be determined.

platform.Platform(aliased=0, terse=0)

Returns a single string identifying the underlying platform with as much useful information as possible.

The output is intended to be human readable rather than machine parseable. It may look different on different platforms and this is intended.

If `aliases` is true, the function will use aliases for various platforms that report system names which differ from their common names, for example SunOS will be reported as Solaris. The `system_alias()` function is used to implement this.

Setting `terse` to true causes the function to return only the absolute minimum information needed to identify the platform.

`platform.processor()`

Returns the (real) processor name, e.g. 'amd64'.

An empty string is returned if the value cannot be determined. Note that many platforms do not provide this information or simply return the same value as for `machine()`. NetBSD does this.

`platform.python_build()`

Returns a tuple (buildno, builddate) stating the Python build number and date as strings.

`platform.python_compiler()`

Returns a string identifying the compiler used for compiling Python.

`platform.python_branch()`

Returns a string identifying the Python implementation SCM branch.

New in version 2.6.

`platform.python_implementation()`

Returns a string identifying the Python implementation. Possible return values are: 'CPython', 'IronPython', 'Jython', 'PyPy'.

New in version 2.6.

`platform.python_revision()`

Returns a string identifying the Python implementation SCM revision.

New in version 2.6.

`platform.python_version()`

Returns the Python version as string 'major.minor.patchlevel'.

Note that unlike the Python `sys.version`, the returned value will always include the patchlevel (it defaults to 0).

`platform.python_version_tuple()`

Returns the Python version as tuple (major, minor, patchlevel) of strings.

Note that unlike the Python `sys.version`, the returned value will always include the patchlevel (it defaults to '0').

`platform.release()`

Returns the system's release, e.g. '2.2.0' or 'NT'. An empty string is returned if the value cannot be determined.

`platform.system()`

Returns the system/OS name, e.g. 'Linux', 'Windows', or 'Java'. An empty string is returned if the value cannot be determined.

`platform.system_alias(system, release, version)`

Returns (system, release, version) aliased to common marketing names used for some systems. It also does some reordering of the information in some cases where it would otherwise cause confusion.

`platform.version()`

Returns the s

ystem's release version, e.g. '#3 on degas'. An empty string is returned if the value cannot be determined.

`platform.uname()`

Fairly portable uname interface. Returns a tuple of strings (system, node, release, version, machine, processor) identifying the underlying platform.

Note that unlike the `os.uname()` function this also returns possible processor information as additional tuple entry.

Entries which cannot be determined are set to "".

Java Platform

```
platform.java_ver(release="", vendor="", vminfo=("", ""), osinfo=("", ""))
```

Version interface for Jython.

Returns a tuple (release, vendor, vminfo, osinfo) with vminfo being a tuple (vm_name, vm_release, vm_vendor) and osinfo being a tuple (os_name, os_version, os_arch). Values which cannot be determined are set to the defaults given as parameters (which all default to "").

Windows Platform

```
platform.win32_ver(release="", version="", csd="", ptype="")
```

Get additional version information from the Windows Registry and return a tuple (release, version, csd, ptype) referring to OS release, version number, CSD level (service pack) and OS type (multi/single processor).

As a hint: ptype is 'Uniprocessor Free' on single processor NT machines and 'Multiprocessor Free' on multi processor machines. The 'Free' refers to the OS version being free of debugging code. It could also state 'Checked' which means the OS version uses debugging code, i.e. code that checks arguments, ranges, etc.

Note This function works best with Mark Hammond's win32all package installed, but also on Python 2.3 and later (support for this was added in Python 2.6). It obviously only runs on Win32 compatible platforms.

Win95/98 specific

```
platform.popen(cmd, mode='r', bufsize=None)
```

Portable popen() interface. Find a working popen implementation preferring win32pipe.popen(). On Windows NT, win32pipe.popen() should work; on Windows 9x it hangs due to bugs in the MS C library.

Mac OS Platform

```
platform.mac_ver(release="", versioninfo=("", ""), machine="")
```

Get Mac OS version information and return it as tuple (release, versioninfo, machine) with versioninfo being a tuple (version, dev_stage, non_release_version).

Entries which cannot be determined are set to ". All tuple entries are strings.

Unix Platforms

```
platform.dist(distname="", version="", id="", supported_dists=('SuSE', 'debian', 'redhat', 'mandrake', ...))
```

This is an old version of the functionality now provided by `linux_distribution()`. For new code, please use the `linux_distribution()`.

The only difference between the two is that `dist()` always returns the short name of the distribution taken from the `supported_dists` parameter.

Deprecated since version 2.6.

```
platform.linux_distribution(distname="", version="", id="", supported_dists=('SuSE', 'debian', 'redhat', 'mandrake', ...), full_distribution_name=1)
```

Tries to determine the name of the Linux OS distribution name.

`supported_dists` may be given to define the set of Linux distributions to look for. It defaults to a list of currently supported Linux distributions identified by their release file name.

If `full_distribution_name` is true (default), the full distribution read from the OS is returned. Otherwise the short name taken from `supported_dists` is used.

Returns a tuple (distname,version,id) which defaults to the args given as parameters. id is the item in parentheses after the version number. It is usually the version codename.

Note This function is deprecated since Python 3.5 and removed in Python 3.8. See alternative like the `distro` package.

New in version 2.6.

```
platform.libc_ver(executable=sys.executable, lib="", version="", chunksize=2048)
```

Tries to determine the libc version against which the file executable (defaults to the Python interpreter) is linked. Returns a tuple of strings (lib, version) which default to the given parameters in case the lookup fails.

Note that this function has intimate knowledge of how different libc versions add symbols to the executable is probably only usable for executables compiled using gcc. The file is read and scanned in chunks of `chunksize` bytes.

2. Using the Python Interpreter

2.1. Invoking the Interpreter

The Python interpreter is usually installed as `/usr/local/bin/python3.8` on those machines where it is available; putting `/usr/local/bin` in your Unix shell's search path makes it possible to start it by typing the command:

```
python3.8
```

to the shell. ¹ Since the choice of the directory where the interpreter lives is an installation option, other places are possible; check with your local Python guru or system administrator. (E.g., `/usr/local/python` is a popular alternative location.)

On Windows machines where you have installed Python from the Microsoft Store, the `python3.8` command will be available. If you have the `py.exe` launcher installed, you can use the `py` command. See Excursus: Setting environment variables for other ways to launch Python.

Typing an end-of-file character (Control-D on Unix, Control-Z on Windows) at the primary prompt causes the interpreter to exit with a zero exit status. If that doesn't work, you can exit the interpreter by typing the following command: `quit()`.

The interpreter's line-editing features include interactive editing, history substitution and code completion on systems that support the GNU Readline library. Perhaps the quickest check to see whether command line editing is supported is typing Control-P to the first Python prompt you get. If it beeps, you have command line editing; see Appendix Interactive Input Editing and History Substitution for an introduction to the keys. If nothing appears to happen, or if `^P` is echoed, command line editing isn't available; you'll only be able to use backspace to remove characters from the current line.

The interpreter operates somewhat like the Unix shell: when called with standard input connected to a tty device, it reads and executes commands interactively; when called with a file name argument or with a file as standard input, it reads and executes a script from that file.

A second way of starting the interpreter is `python -c command [arg] ...`, which executes the statement(s) in command, analogous to the shell's `-c` option. Since Python statements often contain spaces or other characters that are special to the shell, it is usually advised to quote command in its entirety with single quotes.

Some Python modules are also useful as scripts. These can be invoked using `python -m module [arg] ...`, which executes the source file for module as if you had spelled out its full name on the command line.

When a script file is used, it is sometimes useful to be able to run the script and enter interactive mode afterwards. This can be done by passing `-i` before the script.

All command line options are described in [Command line and environment](#).

Argument Passing

When known to the interpreter, the script name and additional arguments thereafter are turned into a list of strings and assigned to the `argv` variable in the `sys` module. You can access this list by executing `import sys`. The length of the list is at least one; when no script and no arguments are given, `sys.argv[0]` is an empty string. When the script name is given as `'-'` (meaning standard input), `sys.argv[0]` is set to `'-'`. When `-c` command is used, `sys.argv[0]` is set to `'-c'`. When `-m module` is used, `sys.argv[0]` is set to the full name of the located module. Options found after `-c` command or `-m module` are not consumed by the Python interpreter's option processing but left in `sys.argv` for the command or module to handle.

Interactive Mode

When commands are read from a tty, the interpreter is said to be in interactive mode. In this mode it prompts for the next command with the primary prompt, usually three greater-than signs (`>>>`); for continuation lines it prompts with the secondary prompt, by default three dots (`...`). The interpreter prints a welcome message stating its version number and a copyright notice before printing the first prompt:

```
$ python3.8
```

```
Python 3.8 (default, Sep 16 2015, 09:25:04)
```

```
[GCC 4.8.2] on linux
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>>
```

Continuation lines are needed when entering a multi-line construct. As an example, take a look at this if statement:

```
>>>
```

```
>>>the_world_is_flat = True
```



```
>>>ifthe_world_is_flat:

...     print("Be careful not to fall off!")
```

Be careful not to fall off!

For more on interactive mode, see Interactive Mode.

2.2. The Interpreter and Its Environment

2.2.1. Source Code Encoding

By default, Python source files are treated as encoded in UTF-8. In that encoding, characters of most languages in the world can be used simultaneously in string literals, identifiers and comments — although the standard library only uses ASCII characters for identifiers, a convention that any portable code should follow. To display all these characters properly, your editor must recognize that the file is UTF-8, and it must use a font that supports all the characters in the file.

To declare an encoding other than the default one, a special comment line should be added as the first line of the file. The syntax is as follows:

```
# -*- coding: encoding -*-
```

where encoding is one of the valid codecs supported by Python.

For example, to declare that Windows-1252 encoding is to be used, the first line of your source code file should be:

```
# -*- coding: cp1252 -*-
```

One exception to the first line rule is when the source code starts with a UNIX “shebang” line. In this case, the encoding declaration should be added as the second line of the file. For example:

```
#!/usr/bin/env python3
```

```
# -*- coding: cp1252 -*-
```

Introduction to Artificial Intelligence

“The science and engineering of making intelligent machines, especially intelligent computer programs”. -John McCarthy-

Artificial Intelligence is an approach to make a computer, a robot, or a product to think how smart human think. AI is a study of how human brain think, learn, decide and work, when it tries to solve problems. And finally this study outputs intelligent software systems. The aim of AI is to improve computer functions which are related to human knowledge, for example, reasoning, learning, and problem-solving.

The intelligence is intangible. It is composed of

- Reasoning
- Learning
- Problem Solving
- Perception
- Linguistic Intelligence

The objectives of AI research are reasoning, knowledge representation, planning, learning, natural language processing, realization, and ability to move and manipulate objects. There are long-term goals in the general intelligence sector.

Approaches include statistical methods, computational intelligence, and traditional coding AI. During the AI research related to search and mathematical optimization, artificial neural networks and methods based on statistics, probability, and economics, we use many tools. Computer science attracts AI in the field of science, mathematics, psychology, linguistics, philosophy and so on.

Trending AI Articles:

- [1. Cheat Sheets for AI, Neural Networks, Machine Learning, Deep Learning & Big Data](#)
- [2. Data Science Simplified Part 1: Principles and Process](#)
- [3. Getting Started with Building Realtime API Infrastructure](#)
- [4. AI & NLP Workshop](#)

Applications of AI:

· Gaming – AI plays important role for machine to think of large number of possible positions based on deep knowledge in strategic games. for example, chess, river crossing, N-queens problems and etc.

Natural Language Processing – Interact with the computer that understands natural language spoken by humans.

- Expert Systems – Machine or software provide explanation and advice to the users.
- Vision Systems – Systems understand, explain, and describe visual input on the computer.
- Speech Recognition – There are some AI based speech recognition systems have ability to hear and express as sentences and understand their meanings while a person talks to it. For example Siri and Google assistant.
- Handwriting Recognition – The handwriting recognition software reads the text written on paper and recognize the shapes of the letters and convert it into editable text.
- Intelligent Robots – Robots are able to perform the instructions given by a human.

Major Goals:

- Knowledge reasoning
- Planning
- Machine Learning
- Natural Language Processing
- Computer Vision
- Robotics

IBM Watson:



“Watson” is an IBM supercomputer that combines Artificial Intelligence (AI) and complex inquisitive programming for ideal execution as a “question answering” machine. The supercomputer is named for IBM’s founder, Thomas J. Watson.

IBM Watson is at the forefront of the new era of computing. At the point when IBM Watson made, IBM communicated that “more than 100 particular techniques are used to inspect perceive sources, find and make theories, find and score affirm, and combination and rank speculations.” recently, the Watson limits have been expanded and the way by which Watson works has been changed to abuse new sending models (Watson on IBM Cloud) and propelled machine learning capacities and upgraded hardware open to architects and authorities. It isn’t any longer completely a request answering figuring system arranged from Q&A joins yet can now ‘see’, ‘hear’, ‘read’, ‘talk’, ‘taste’, ‘translate’, ‘learn’ and ‘endorse’.

Machine Learning

Introduction:

Machine learning is a subfield of artificial intelligence (AI). The goal of machine learning generally is to understand the structure of data and fit that data into models that can be understood and utilized by people.

Although machine learning is a field within computer science, it differs from traditional computational approaches. In traditional computing, algorithms are sets of explicitly programmed instructions used by computers to calculate or problem solve. Machine learning algorithms instead allow for computers to train on data inputs and use statistical analysis in order to output values that fall within a specific range. Because of this, machine learning facilitates computers in building models from sample data in order to automate decision-making processes based on data inputs.

Any technology user today has benefitted from machine learning. Facial recognition technology allows social media platforms to help users tag and share photos of friends. Optical character recognition (OCR) technology converts images of text into movable type. Recommendation engines, powered by machine learning, suggest what movies or television shows to watch next based on user preferences. Self-driving cars that rely on machine learning to navigate may soon be available to consumers.

Machine learning is a continuously developing field. Because of this, there are some considerations to keep in mind as you work with machine learning methodologies, or analyze the impact of machine learning processes.

In this tutorial, we'll look into the common machine learning methods of supervised and unsupervised learning, and common algorithmic approaches in machine learning, including the k-nearest neighbor algorithm, decision tree learning, and deep learning. We'll explore which programming languages are most used in machine learning, providing you with some of the positive and negative attributes of each. Additionally, we'll discuss biases that are perpetuated by machine learning algorithms, and consider what can be kept in mind to prevent these biases when building algorithms.

Machine Learning Methods:

In machine learning, tasks are generally classified into broad categories. These categories are based on how learning is received or how feedback on the learning is given to the system developed.

Two of the most widely adopted machine learning methods are **supervised learning** which trains algorithms based on example input and output data that is labeled by humans, and **unsupervised learning** which provides the algorithm with no labeled data in order to allow it to find structure within its input data. Let's explore these methods in more detail.

Supervised Learning:

In supervised learning, the computer is provided with example inputs that are labeled with their desired outputs. The purpose of this method is for the algorithm to be able to “learn” by comparing its actual output with the “taught” outputs to find errors, and modify the model accordingly. Supervised learning therefore uses patterns to predict label values on additional unlabeled data.

For example, with supervised learning, an algorithm may be fed data with images of sharks labeled as `fish` and images of oceans labeled as `water`. By being trained on this data, the supervised learning algorithm should be able to later identify unlabeled shark images as `fish` and unlabeled ocean images as `water`.

A common use case of supervised learning is to use historical data to predict statistically likely future events. It may use historical stock market information to anticipate upcoming fluctuations, or be employed to filter out spam emails. In supervised learning, tagged photos of dogs can be used as input data to classify untagged photos of dogs.

Unsupervised Learning:

In unsupervised learning, data is unlabeled, so the learning algorithm is left to find commonalities among its input data. As unlabeled data are more abundant than labeled data, machine learning methods that facilitate unsupervised learning are particularly valuable.

The goal of unsupervised learning may be as straightforward as discovering hidden patterns within a dataset, but it may also have a goal of feature learning, which allows the computational machine to automatically discover the representations that are needed to classify raw data.

Unsupervised learning is commonly used for transactional data. You may have a large dataset of customers and their purchases, but as a human you will likely not be able to make sense of what similar attributes can be drawn from customer profiles and their types of purchases. With this data fed into an unsupervised learning algorithm, it may be determined that women of a certain age range who buy unscented soaps are likely to be pregnant, and therefore a marketing campaign related to pregnancy and baby products can be targeted to this audience in order to increase their number of purchases.

Without being told a “correct” answer, unsupervised learning methods can look at complex data that is more expansive and seemingly unrelated in order to organize it in potentially meaningful ways. Unsupervised learning is often used for anomaly detection including for fraudulent credit card purchases, and recommender systems that recommend what products to buy next. In unsupervised learning, untagged photos of dogs can be used as input data for the algorithm to find likenesses and classify dog photos together.

Approaches:

As a field, machine learning is closely related to computational statistics, so having a background knowledge in statistics is useful for understanding and leveraging machine learning algorithms.

For those who may not have studied statistics, it can be helpful to first define correlation and regression, as they are commonly used techniques for investigating the relationship among quantitative variables. **Correlation** is a measure of association between two variables that are not designated as either dependent or independent. **Regression** at a basic level is used to examine the relationship between one

dependent and one independent variable. Because regression statistics can be used to anticipate the dependent variable when the independent variable is known, regression enables prediction capabilities.

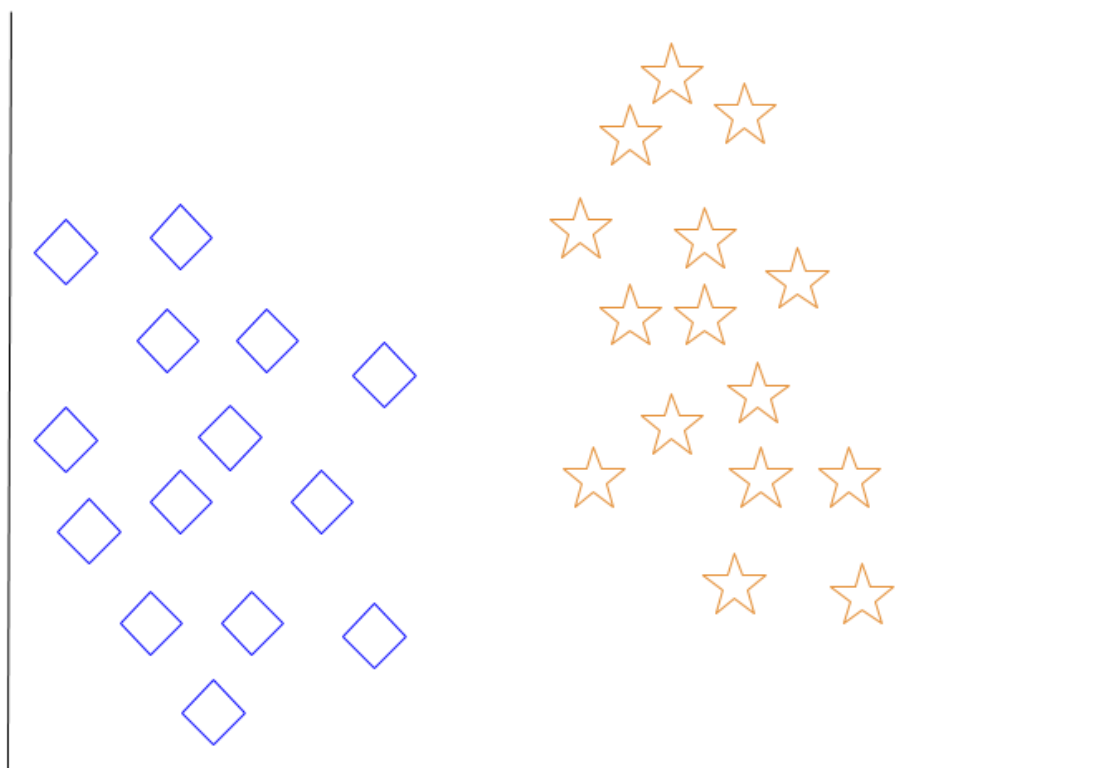
Approaches to machine learning are continuously being developed. For our purposes, we'll go through a few of the popular approaches that are being used in machine learning at the time of writing.

k-nearest neighbor:

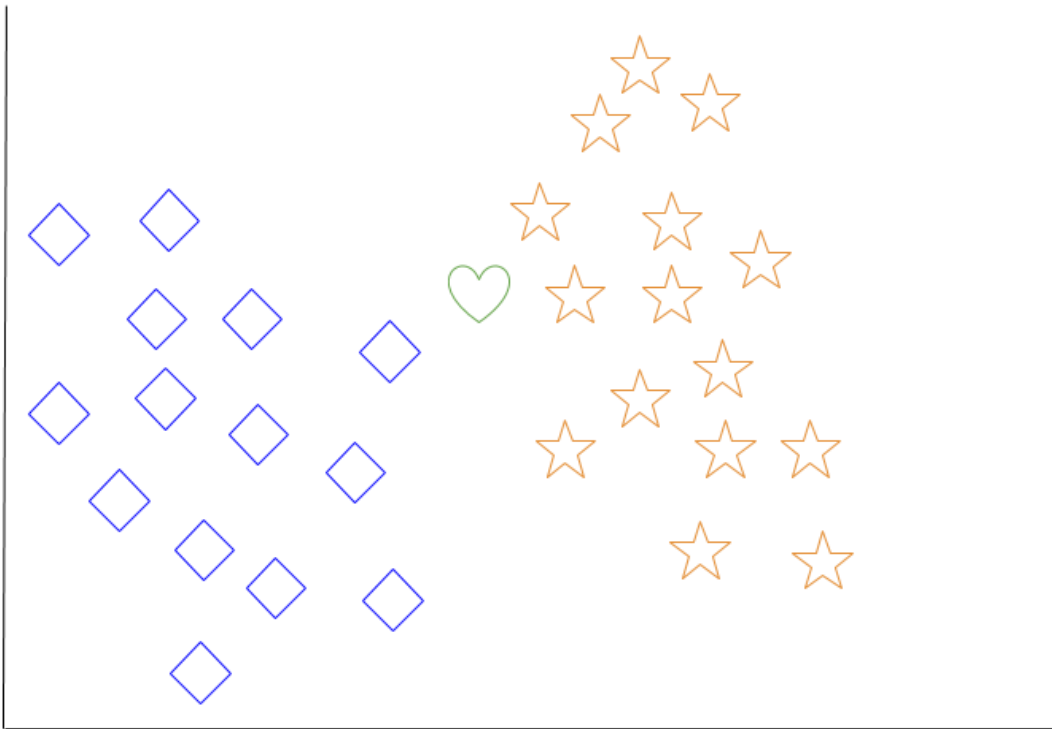
The k-nearest neighbor algorithm is a pattern recognition model that can be used for classification as well as regression. Often abbreviated as k-NN, the **k** in k-nearest neighbor is a positive integer, which is typically small. In either classification or regression, the input will consist of the k closest training examples within a space.

We will focus on k-NN classification. In this method, the output is class membership. This will assign a new object to the class most common among its k nearest neighbors. In the case of $k = 1$, the object is assigned to the class of the single nearest neighbor.

Let's look at an example of k-nearest neighbor. In the diagram below, there are blue diamond objects and orange star objects. These belong to two separate classes: the diamond class and the star class.

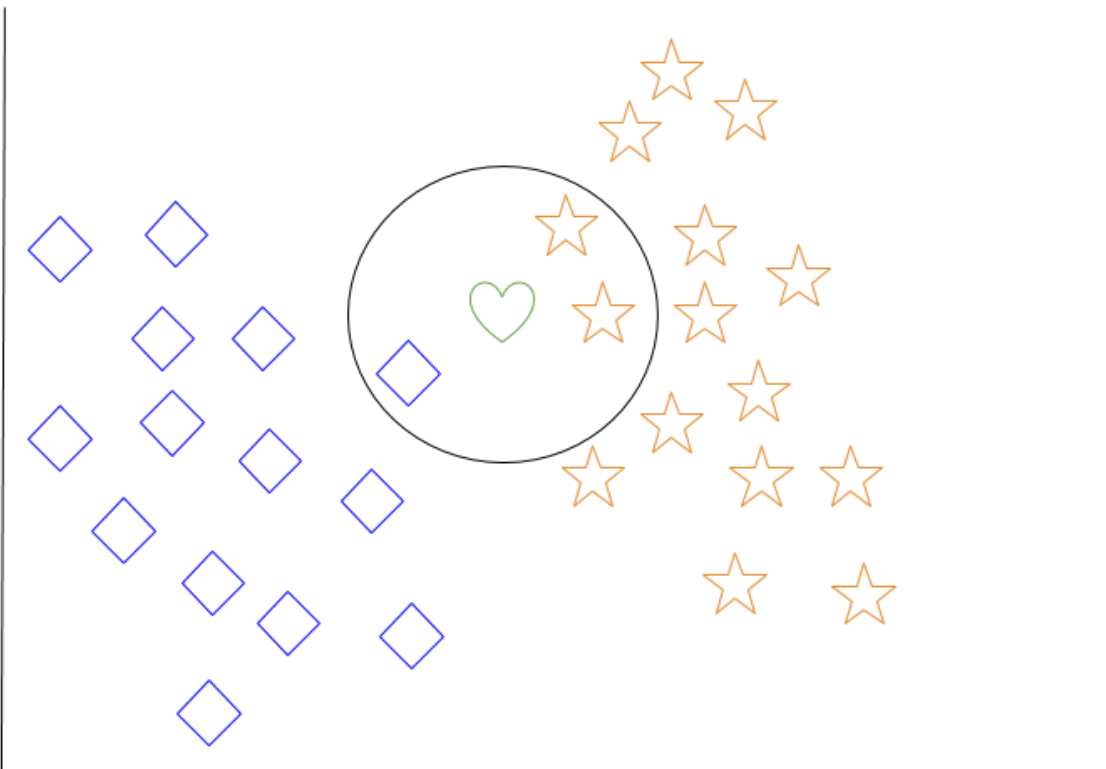


When a new object is added to the space — in this case a green heart — we will want the machine learning algorithm to classify the heart to a certain class.



When we choose $k = 3$, the algorithm will find the three nearest neighbors of the green heart in order to classify it to either the diamond class or the star class.

In our diagram, the three nearest neighbors of the green heart are one diamond and two stars. Therefore, the algorithm will classify the heart with the star class.



Among the most basic of machine learning algorithms, k-nearest neighbor is considered to be a type of “lazy learning” as generalization beyond the training data does not occur until a query is made to the system.

Decision Tree Learning:

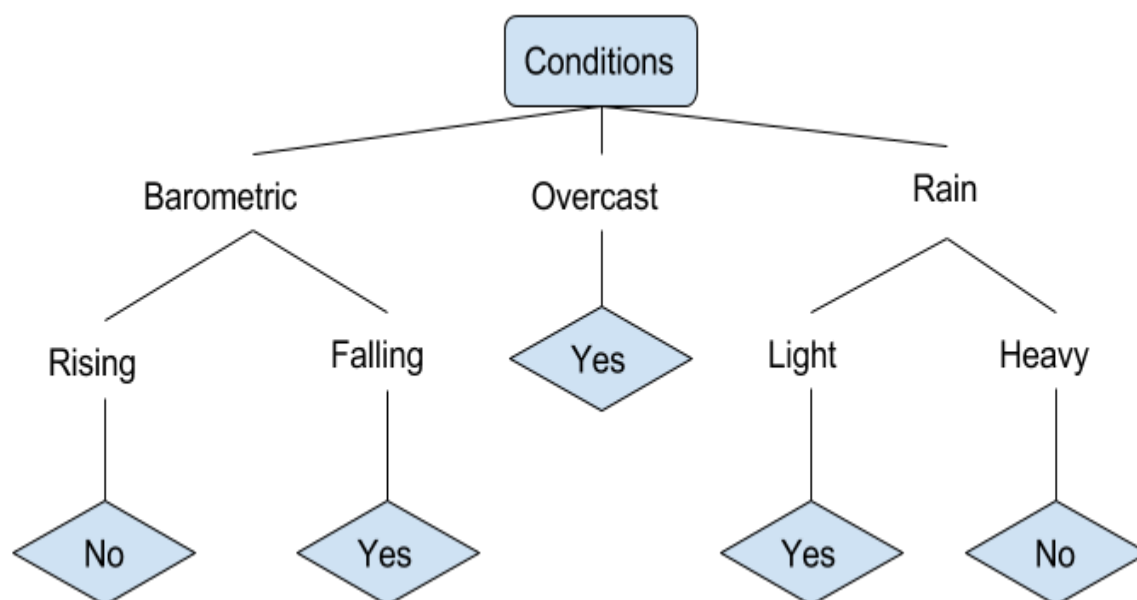
For general use, decision trees are employed to visually represent decisions and show or inform decision making. When working with machine learning and data mining, decision trees are used as a predictive model. These models map observations about data to conclusions about the data’s target value.

The goal of decision tree learning is to create a model that will predict the value of a target based on input variables.

In the predictive model, the data’s attributes that are determined through observation are represented by the branches, while the conclusions about the data’s target value are represented in the leaves.

When “learning” a tree, the source data is divided into subsets based on an attribute value test, which is repeated on each of the derived subsets recursively. Once the subset at a node has the equivalent value as its target value has, the recursion process will be complete.

Let’s look at an example of various conditions that can determine whether or not someone should go fishing. This includes weather conditions as well as barometric pressure conditions.



In the simplified decision tree above, an example is classified by sorting it through the tree to the appropriate leaf node. This then returns the classification associated with the particular leaf, which in this case is either a **Yes** or a **No**. The tree classifies a day's conditions based on whether or not it is suitable for going fishing.

A true classification tree data set would have a lot more features than what is outlined above, but relationships should be straightforward to determine. When working with decision tree learning, several determinations need to be made, including what features to choose, what conditions to use for splitting, and understanding when the decision tree has reached a clear ending.

Introduction to Deep Learning

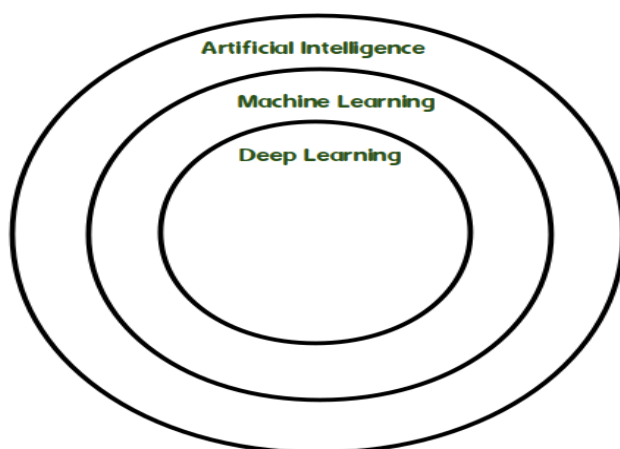
What is deep learning

Deep learning is a branch of machine learning which is completely based on artificial neural networks, as neural network is going to mimic the human brain so deep learning is also a kind of mimic of human brain. In deep learning, we don't need to explicitly program everything. The concept of deep learning is not new. It has been around for a couple of years now. It's on hype nowadays because earlier we did not have that much processing power and a lot of data. As in the last 20 years, the processing power increases exponentially, deep learning and machine learning came in the picture.

A formal definition of deep learning is- neurons

Deep learning is a particular kind of machine learning that achieves great power and flexibility by learning to represent the world as a nested hierarchy of concepts, with each concept defined in relation to simpler concepts, and more abstract representations computed in terms of less abstract ones.

The question here is how do we recreate these neurons in a computer. So, we create an artificial structure called an artificial neural net where we have nodes or neurons. We have some neurons for input value and some for output value and in between, there may be lots of neurons interconnected in the hidden layer.



SDLC :

SDLC stands for Software Development Life Cycle. A Software Development Life Cycle is essentially a series of steps, or phases, that provide a model for the development and lifecycle management of an application or piece of software.

SDLC is the process consisting of a series of planned activities to develop or alter the software products.

Benefits of the SDLC Process:

The intent of a SDLC process is to help produce a product that is cost-efficient, effective, and of high quality. Once an application is created, the SDLC maps the proper deployment and decommissioning of the software once it becomes a legacy. The SDLC methodology usually contains the following stages: Analysis (requirements and design), construction, testing, release, and maintenance (response).

1.Requirements Gathering:

In this phase we gather all the requirements from the client, i.e. what are the client expected input, output.....

2.Analysis:

In this phase based upon the client requirements we prepare one documentation is called “High Level Design Document”. It contains Abstract, Functional Requirements, Non Functional Requirements, Existing System, Proposed System, SRS,.....

3.Design:

It is difficult to understand the High Level Design Document for all the members, so to understand easily we use “Low Level Design Document”. To design this document we use UML (Unified Modeling Language).

In this we have Use case, Sequence, Collaboration.....

4.Coding:

In this phase we develop the coding module by module. After developing all the modules we integrate them.

5.Testing:

After developing we have to check whether client requirements are satisfied or not. If not we are again going to develop.

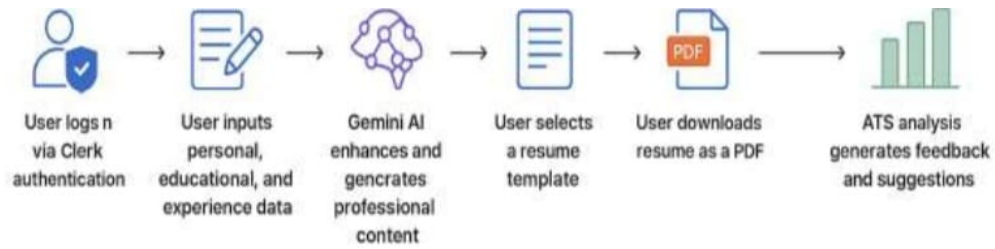
6.Implementation:

In testing phase if client requirements are satisfied, we go for implementation. i.e. we need to deploy the application in some server.

7.Maintenance:

After deployment, if at all any problems come from the client side; we are providing maintenance for that application.

SCREENSHOTS



V. RESULTS AND DISCUSSION

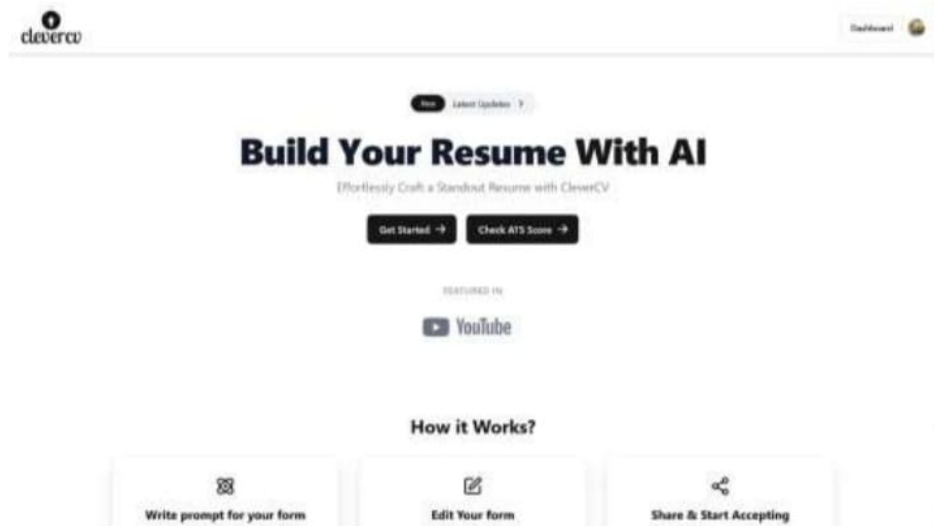


Figure 1: Home Page

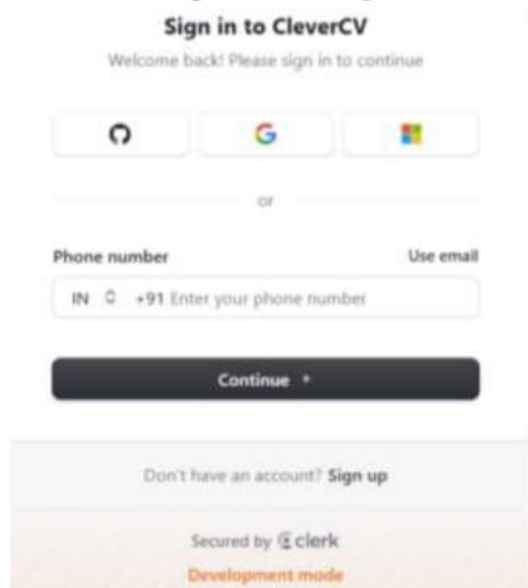
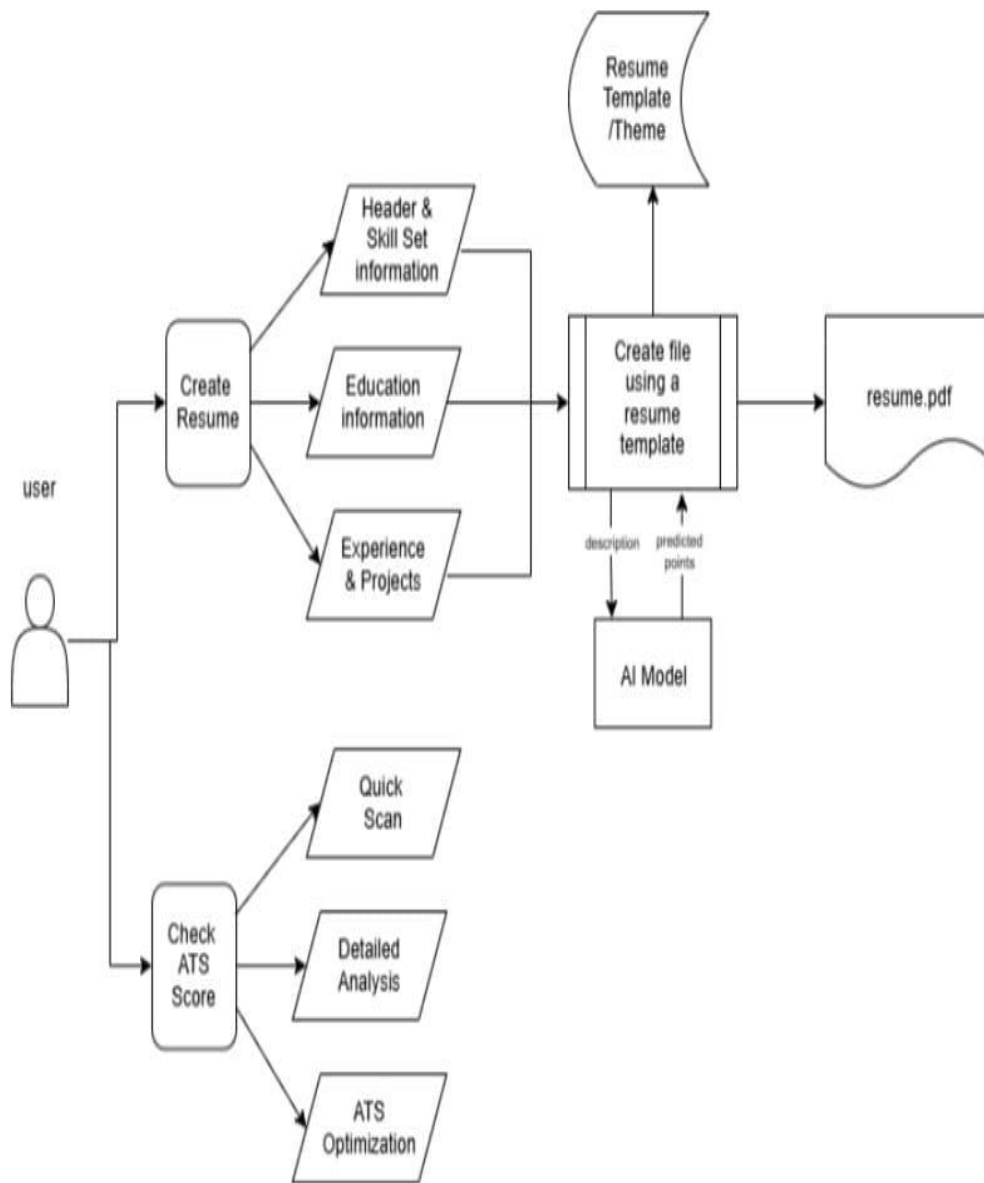


Figure 2: Login Page



ResumeATS Pro

Optimize Your Resume for ATS and Land Your Dream Job

Upload your resume (PDF)

Drag and drop file here
or
Choose File

Choose File

Upload CV.pdf (100 KB)

X

Enter the job description (optional)

Job Title: AWS Solutions Architect
Location: Pune
Job Type: Full-Time

Choose analysis type:

- ☐ Quick Scan
- ☐ Detailed Analysis
- ☒ ATS Optimization

Analyze Resume

Figure 6: ATS Score Checker Page

Resume Optimization for Atharva Jadhav

1. Keywords from Job Description:

The provided job description is minimal. However, we've identified relevant keywords based on the typical requirements of an AWS Solutions Architect role:

- AWS Solutions Architect
- Cloud Computing
- Cloud-based solutions
- Infrastructure as Code (IaC) - if applicable, add Pulumi projects
- Cloud Security
- Compliance (e.g., HIPAA, SOC 2, PCI DSS) - specify if applicable
- Networking (VPC, Route Tables, Security Groups)
- Automation (if relevant)
- Agile (if relevant)
- Scripting (if relevant)

2. Formatting and Punctuation for ATS Readability:

- Use a consistent format: Use a standard, professional font (e.g., Arial, Calibri, Times New Roman) in a size easily readable (10-12pt).
- Remove unnecessary spacing and line breaks: The current resume has inconsistent spacing. Minimize a clean, consistent layout.
- Use bullet points consistently: All bullet points should have the same formatting.
- Quantify accomplishments: Use numbers to demonstrate impact whenever possible (e.g., "Reduced server costs by 15%").
- Structure: The current structure is acceptable but can be improved. A suggested order is: Contact Information, Summary, Skills, Experience, Education, Projects.
- Header: Make the header bold and large.

3. Improving Keyword Density without Keyword Stuffing:

Instead of simply adding keywords, integrate them naturally within the context of your accomplishments. For example:

- Instead of: "Responsible for architecting and implementing cloud-based solutions on AWS."
- Try: "Designed and implemented secure, scalable cloud-based solutions on AWS, ensuring high availability and compliance with industry standards."

4. Rating (out of 100):

- Impact: 6/10 (Could be significantly improved with stronger quantifiable results)
- Brevity: 7/10 (Generally concise, but some sections could be shortened)
- Style: 5/10 (Formatting is inconsistent, needs proofreading for typos and spacing)
- Structure: 7/10 (Logical structure, but could be improved with stronger section headings)
- Skills: 7/10 (Relevant skills are listed, but could benefit from categorization and proficiency levels)

5. Review of Major Sections:

- Summary: Weak. Needs to be rewritten to be more compelling and result-oriented.
- Experience: Adequate but lacks quantifiable achievements. Needs significant improvement using the STAR method.
- Education: Okay, but the sentence explaining the ongoing degree is redundant.
- Skills: Good starting point, but requires better organization and potential addition of proficiency levels.
- Projects: Decent, but lacks depth and could benefit from showcasing more complex projects.

6. Overall ATS Score (out of 100):

- Keyword Optimization (30/30): Strong use of relevant keywords like "AWS," "Solutions Architect," "Cloud Computing," etc.
- Formatting and Readability (15/20): Inconsistent formatting and minor typos detract from readability.
- Content and Quantifiable Results (25/30): Content is relevant, but lacks sufficient quantifiable results to showcase impact.
- Experience and Skills Matching (10/20): Experience and skills match the target job profile well.

Total ATS Score: 80/100

This score reflects the resume's strengths but also highlights the areas where significant improvements can be made to increase its effectiveness. Focusing on quantifying achievements and using the STAR method in the experience section would drastically improve the impact and ATS score.

Figure 7: Given Feedback and ATS Score

Summary

Add Summary for your job title

Add Summary

Generate Resume AI

Done

Suggestions

Level Senior

Senior AWS Solutions Architect with 5+ years of experience designing, implementing, and managing cloud-based solutions. Proven ability to lead projects from inception to completion, consistently exceeding expectations. Expertise in various AWS services including EC2, S3, RDS, and Lambda. Strong problem-solving skills and a passion for innovation.

Level Mid

Results-oriented AWS Solutions Architect with 3-5 years of experience in designing and deploying scalable and reliable cloud architectures. Proficient in implementing security best practices and cost optimization strategies. Experience with DevOps methodologies and CI/CD pipelines. Excellent communication and collaboration skills.

Level Fresh

Highly motivated and enthusiastic recent graduate with a strong understanding of cloud computing principles and AWS services. Eager to apply academic knowledge and learn practical skills in a professional environment. A fast learner with a proven ability to adapt quickly to new technologies and challenges. Seeking an entry-level position as an AWS Solutions Architect.

Figure 3: Summary Generation Page

Projects

Add New Project

Project Name

Description

Tech Stack

+ Add New Projects

- Remove

Done

Atharva Jadhav

AWS Solutions Architect

Resume AI

Highly motivated and enthusiastic recent graduate with a strong understanding of cloud computing principles and AWS services. Eager to apply academic knowledge and learn practical skills in a professional environment. A fast learner with a proven ability to adapt quickly to new technologies and challenges. Seeking an entry-level position as an AWS Solutions Architect.

Professional Experience

Associate Solutions Architect Intern

Radical Technologies Pune, Maharashtra

2024-05-15 To 2024-11-15

- Assisted senior architects in designing and implementing cloud-based solutions on AWS.
- Developed and maintained documentation for cloud infrastructure projects.
- Contributed to the improvement of existing cloud infrastructure projects.
- Collaborated with cross-functional teams to ensure successful project delivery.
- Learned and applied industry best practices for cloud security and compliance.

Education

Savitribai Phule Pune University

Bachelor of Engineering (B.E.) in Computer Engineering

2021-12-12 - 2023-05-15

I am pursuing a Bachelor of Engineering (B.E.) in Computer Engineering from Savitribai Phule Pune University (SPPU), with a strong foundation in programming, web development, cloud computing, and software engineering principles.

ISCED

HSC in PCM

2020-06-15 - 2021-08-15

Completed Higher Secondary Certificate (HSC) with a focus on Science, showcasing a solid understanding of core subjects while developing strong analytical and problem-solving skills.

Skills

AWS

100%

100%

Python

100%

100%

JavaScript

100%

100%

Projects

AWS VPC to VPC Peering

Implemented VPC-to-VPC peering between AWS VPCs, managing route tables and security groups for robust and efficient cross-VPC communication.

Tech Stack: AWS

Figure 4: Resume Creation Page

Atharva Jadhav

AWS Solutions Architect

Resume AI

Highly motivated and enthusiastic recent graduate with a strong understanding of cloud computing principles and AWS services. Eager to apply academic knowledge and learn practical skills in a professional environment. A fast learner with a proven ability to adapt quickly to new technologies and challenges. Seeking an entry-level position as an AWS Solutions Architect.

Professional Experience

Associate Solutions Architect Intern

Radical Technologies Pune, Maharashtra

2024-05-15 To 2024-11-15

- Assisted senior architects in designing and implementing cloud-based solutions on AWS.
- Developed and maintained documentation for cloud infrastructure projects.
- Contributed to the improvement of existing cloud infrastructure projects.
- Collaborated with cross-functional teams to ensure successful project delivery.
- Learned and applied industry best practices for cloud security and compliance.

Education

Savitribai Phule Pune University

Bachelor of Engineering (B.E.) in Computer Engineering

2021-12-12 - 2023-05-15

I am pursuing a Bachelor of Engineering (B.E.) in Computer Engineering from Savitribai Phule Pune University (SPPU), with a strong foundation in programming, web development, cloud computing, and software engineering principles.

ISCED

HSC in PCM

2020-06-15 - 2021-08-15

Completed Higher Secondary Certificate (HSC) with a focus on Science, showcasing a solid understanding of core subjects while developing strong analytical and problem-solving skills.

Skills

AWS

100%

100%

Python

100%

100%

JavaScript

100%

100%

Projects

AWS VPC to VPC Peering

Implemented VPC-to-VPC peering between AWS VPCs, managing route tables and security groups for robust and efficient cross-VPC communication.

Tech Stack: AWS

Figure 5: Generated Resume

9.CONCLUSION

The proposed AI-powered resume generation system addresses a critical need in today's digital recruitment landscape. By integrating NLP and machine learning, it empowers job seekers to create ATS-compliant resumes that are optimized for specific job roles. The system not only enhances resume quality through real-time feedback and keyword analysis but also assists users in identifying appropriate roles based on their skills and experiences.

Independent evaluations confirm its effectiveness in helping diverse users navigate the complex job application process. While ethical considerations regarding authenticity must be acknowledged, the system lays a strong foundation for the development of intelligent career advisory platforms. In conclusion, this research highlights the transformative role of AI in resume creation and job matching, offering a promising direction for future employment technologies.

AI-powered resume builders significantly enhance job applications by leveraging artificial intelligence to analyze job descriptions, optimize keywords, and improve overall resume structure and clarity. They save time, reduce human errors, and help candidates tailor resumes to specific roles, increasing compatibility with Applicant Tracking Systems (ATS). While human judgment is still essential for personalization and authenticity, AI-driven tools serve as a powerful support system, enabling job seekers to present their skills and experiences more effectively in a competitive job market.

AI-powered resume builders play a crucial role in modern job applications by using artificial intelligence to create well-structured, professional, and ATS-friendly resumes. These tools analyze job descriptions, identify relevant keywords, and align candidate skills with employer requirements, thereby increasing the chances of shortlisting. They also provide real-time suggestions for improving grammar, formatting, and content quality, helping candidates present their profiles more effectively.

Moreover, AI resume builders save time and reduce the stress involved in manual resume creation, especially for freshers and job seekers applying to multiple roles. While they cannot fully replace human creativity and personal judgment, when used wisely, AI-powered tools act as valuable assistants that enhance resume quality, improve visibility in recruitment systems, and support candidates in achieving better career opportunities.

10.REFERENCES

- [1] Wu, Q., Liu, Y., Li, Q., Jin, S., & Li, F. (2020). The Application of AI in ATS Optimization: A Comprehensive Review. *Journal of AI in Recruitment*, 12(3), 215-230.
- [2] Ghosh, A., Sen, D., & Saha, R. (2019). AI-Driven Content Generation for Resume Optimization. *Journal of Artificial Intelligence Applications*, 14(4), 324-335
- [3] Lowe, D. G. (2021). Enhancing Applicant Tracking Systems Through AI Algorithms. *IEEE Transactions on Human Resources Technology*, 19(6), 123-140.
- [4] Smith, J., Johnson, A., & Williams, K. (2020). Natural Language Processing for Resume Generation: Enhancing Resume Quality Using GPT-3. *Journal of Computational Linguistics*, 15(3), 234-256.
- [5] Sharma, R., & Patel, S. (2021). AI-Powered Resume Builders: Streamlining Content Generation and Personalization. *International Journal of Artificial Intelligence Research*, 9(2), 112-129.
- [6] Brown, L., & Nguyen, T. (2019). The Impact of Applicant Tracking Systems on Job Search: An AI-Based Approach. *Human Resource Technology Quarterly*, 5(1), 87-103.
- [7] Jones, M., Taylor, D., & Reed, S. (2022). AI in Resume Optimization: Ensuring ATS Compatibility. *Proceedings of the International Conference on Artificial Intelligence and Job Automation*, 32(4), 178-192
- [8] Kumar, S., & Lee, J. (2020). Multilingual Resume Builders Using AI: Addressing Linguistic Diversity in Job Applications. *Journal of Multilingual Computing*, 12(2), 65-81.
- [9] Li, H., & Roberts, P. (2021). AI for Accessible Resume Creation: Enhancing User Experience for Non Native Speakers and Individuals with Disabilities. *Accessibility and AI*, 7(3), 201-219.